Network Applications, Client-Server and P2P

Lecture 03

Computer Networks and Distributed Programming

Where the application layer sits

We have the whole stack. Now we open the layer at the top, the one the user sees.

- An application does not move data: it hands data down and gets it back at the other end.
- What the transport layer offers upward is a reliable data stream — nothing lost, nothing out of order.
- Two applications on two hosts behave as if they talked directly, provided both follow the same protocol.

So what is left for the application programmer to do?

Everything above that stream: which messages exist, what they mean, and who speaks first.

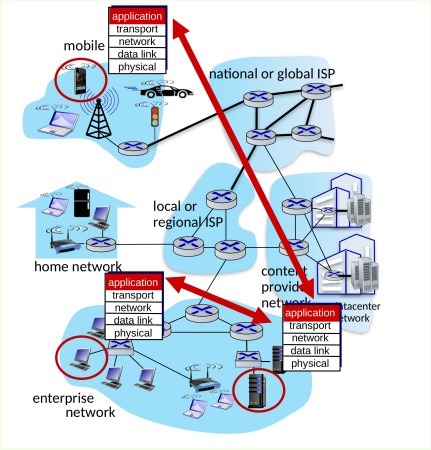
A network application is programs on end systems

Network application

Several programs, running on different end systems, that communicate with each other over the network.

The Web is two distinct programs: the browser on the user's host — desktop, laptop, tablet, phone — and the Web server on the server host.

Both sit at the edge. Between them there is no third program of ours.



Applications you already use

An application is a program running on a device; what the user gets out of it is a service.

- the Web
- e-mail and text messaging
- social networking
- Internet search
- remote login
- streaming of stored video
- real-time video conferencing
- voice over IP
- multi-user network games
- P2P file sharing

They all run over the same network, and every one of them asks it for something different.

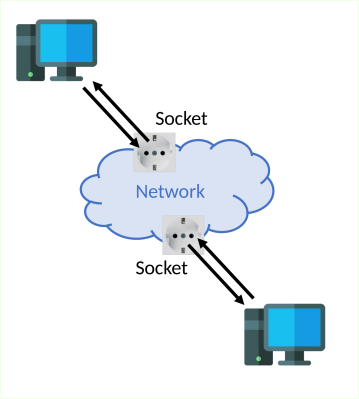
You do not write software for the core

Writing a network application means writing programs that

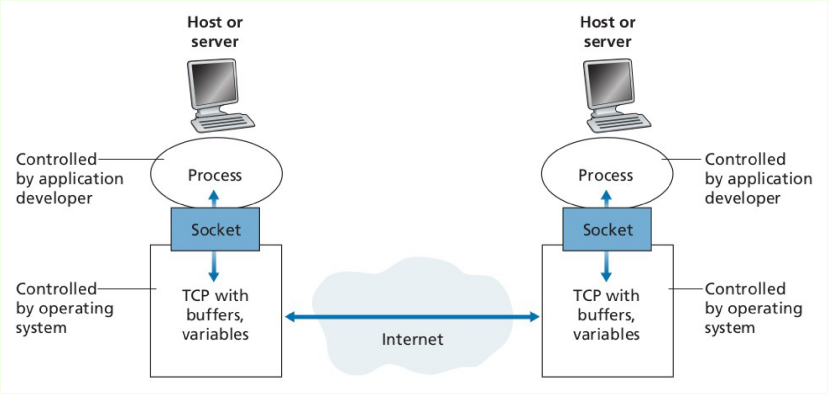
- run on different end systems, possibly in different languages and under different operating systems;
- communicate over the network, following an agreed protocol, through a library — the sockets.

Nothing has to be written for routers or switches. They do not run user applications, and would not let you install one.

That restriction is not a limitation: it is why new applications appear so fast.



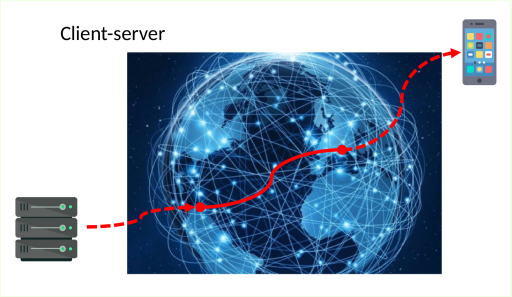
Processes, sockets and the transport protocol



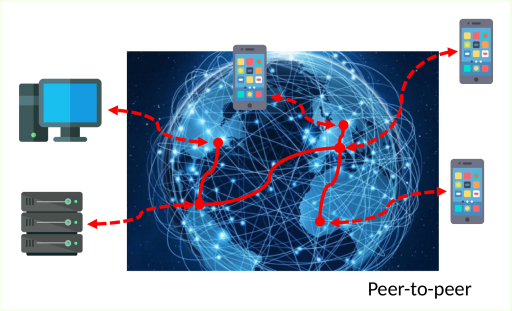
- Two processes communicate, each reaching the network through a socket.
- Below the socket sits the transport protocol, with buffers and variables the application never touches.

Two application architectures

The application architecture is designed by the application developer, and decides how the application is spread over the end systems. Two dominate.



Client-server: the nodes are different, and one of them is always on



Peer-to-peer: the nodes are alike, and connected on and off

What defines client-server

- There is an always-on host, the server, serving requests from many other hosts.
- The server has a fixed, well-known address, so a client can always reach it.
- Clients do not talk to each other: two browsers never exchange anything directly.
- The client speaks first. The server waits to be contacted, and answers.

Consequence

Everything the application does goes through the server, which is therefore both the limit to how far it scales and the single point of failure.

One server is rarely one machine

A single host cannot keep up with the requests of a popular service, so many servers are used — and the client never finds out.

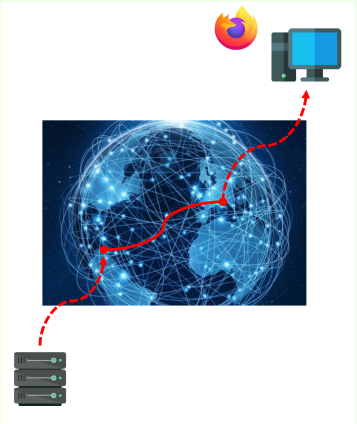
- Data center: a very large number of hosts in one place, to be powered, cooled and maintained.
- Distributed servers: machines scattered around the world, to be interconnected and kept coordinated.
- Distributed data centers: both at once, which is what the large providers actually run.

And the provider keeps paying, month after month, for interconnection and for the bandwidth leaving those buildings.

The Web is the standard case

- An always-on Web server receives requests from the browsers running on the client hosts.
- For each request for an object it responds by sending that object back.
- It is reachable: it has an address the clients can name, and it is up when they ask.

Request, response, and a server that was there before anyone asked: that is the whole shape of it.



Peer-to-peer: self-scalability and its price

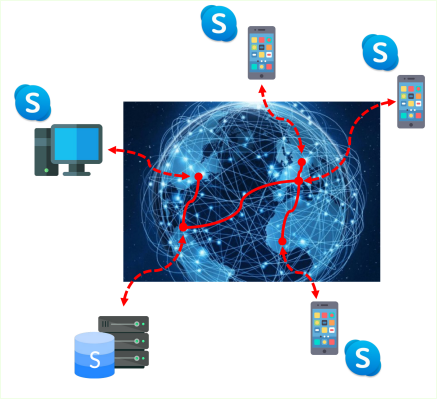
In a P2P architecture there is minimal, or no, reliance on always-on servers: the peers are the users' own machines, connected on and off.

- Each peer generates load by asking for a file, and adds capacity by serving it to others.
- So capacity grows together with demand. This is self-scalability, and no client-server design has it.
- It is also cheap: no server infrastructure to build, no server bandwidth to pay.
- The price is security, performance and reliability — nobody owns the machines involved.

Pure P2P is rare

Most real applications are hybrid: a server for what has to be central, direct exchange for everything else.

- In instant messaging a server tracks where the users are; the messages go host to host.
- In file sharing a server may also index the available files, to make searching fast.
- Telephony, video conferencing and file sharing are the usual cases.



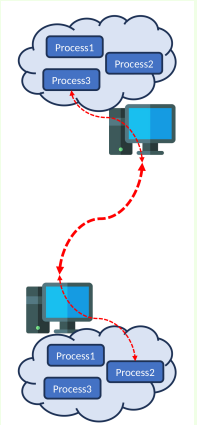
Processes communicate, and take roles

What communicates is a process, not a program and not a machine — and the two may run under different operating systems.

Client and server, precisely

In a session between two processes, the one that initiates the contact is the client; the one that waits to be contacted is the server.

- In file sharing the peer downloading is the client, the one uploading is the server.
- So in P2P one process is both, and changes role from one session to the next.



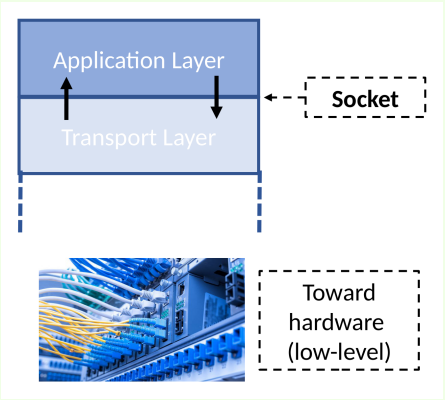
The socket

Socket

The software interface between the application layer and the transport layer inside a host. A process sends messages into it and receives messages from it.

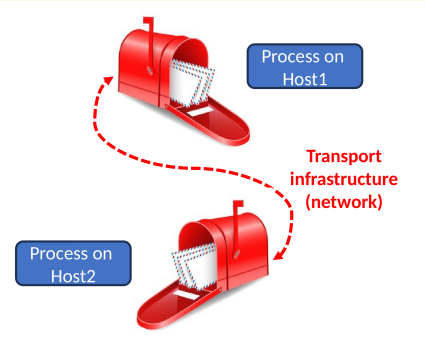
It is also called the API between the application and the network, because it is the interface network programs are built against.

It is the one you will be writing C against.



The mailbox analogy

A process is a house and its socket is the door. The sender pushes the message out of its own door and assumes something outside will carry it.



It arrives through the receiver's door, and only then does the receiving process act on it.

What the developer controls, and what not

Everything on the application side of the socket is yours. On the transport side you have almost nothing.

- You choose the transport protocol — when there is a choice to make.
- You may set a few parameters: buffer sizes, maximum segment size.
- That is all. From then on the application is built on the services that protocol provides.

Why this matters

The choice is made once, at the beginning, and everything else follows from it. It is a design decision, not a configuration detail.

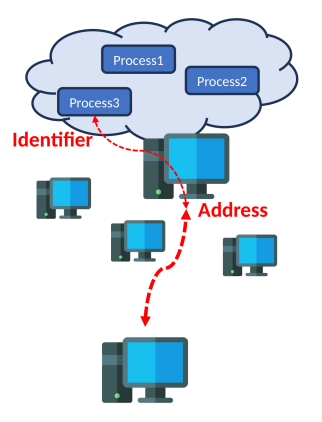
Addressing a process takes two things

To send postal mail you need an address. To send a message to a process running on another host you need two things.

1. An address of the host, to find the right machine on the network.
2. An identifier of the process, to find the right program inside that machine.

The second one is needed because a host runs many network applications at once.

The address is a network layer matter: it is what routing works on. The identifier belongs to the transport layer.

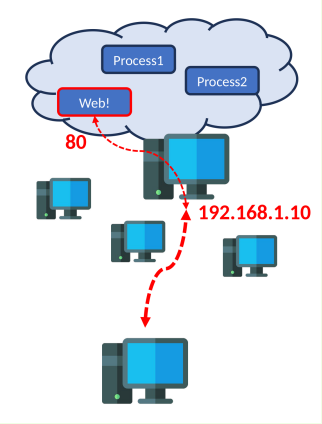


IP address and port number

In the Internet the host is identified by its IP address, a 32-bit quantity, and the process by a port number.

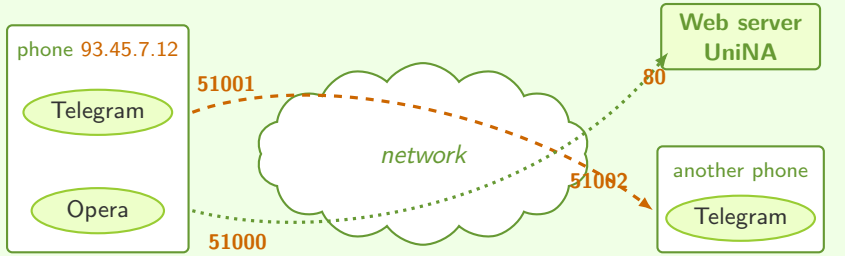
- Popular applications sit on conventional ports: a Web server on 80, a mail server on 25.
- That convention is what lets a client contact a service it has never seen before.
- The complete list of well-known ports is published by IANA (Internet Assigned Numbers Authority).

Without the convention every client would need to be told, in advance, where to knock.



Two applications on one host

A phone with the messaging app and the browser both open, talking to two different destinations at the same time.



Both flows leave from the same address. What keeps them apart, at either end, is the port.

An address belongs to an interface

A laptop is on Ethernet and on Wi-Fi at once. How many addresses does it have?

Two, one per interface. An IP address does not name a machine: it names one attachment point to one network.

- For the network layer that host counts as two distinct hosts, reached along two different paths.
- Unplug the cable and one of the two addresses stops answering, while the other keeps working.

We will come back to this when addresses stop belonging to machines and start belonging to networks.

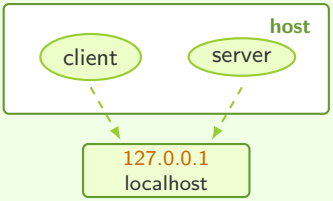
IPv4, and the address that means “here”

IPv4 address

32 bits, written as four decimal numbers between 0 and 255 separated by dots — one number per byte.

127.0.0.1 is special: it does not name a remote machine, it names this one. Two local processes talk through it exactly as they would across the network, and nothing ever leaves the host.

It is how you will test a client against your own server, on one machine, before there is a second one.



nothing reaches the wire

A name is not an address

Nobody types 143.225.161.30. People type `www.unina.it`, and the two designate the same Web server.

- The name is for people: readable, stable, and it survives a change of machine underneath.
- The address is what the network layer works on: it is what routing needs in order to deliver.
- DNS: the Domain Name System translates one into the other, and it is a distributed database: no single machine holds it all.

The DNS is itself an application running over the network, and we will open it shortly.

What you can ask of the transport layer

A transport protocol can offer an application four things. Choosing a protocol means choosing among them.

- Reliable data transfer: what one end sends reaches the other, complete, correct and in order.
- Throughput: a guaranteed rate at which the sender can deliver bits to the receiver.
- Timing: every bit put into the socket arrives within a bounded delay.
- Security: the data is encrypted on the way out and decrypted on the way in.

Which of the four does your application actually need?

What real applications need

Application	Data loss	Throughput	Time-sensitive
File transfer	no loss	elastic	no
E-mail	no loss	elastic	no
Web documents	no loss	elastic, a few kbps	no
Internet telephony	loss-tolerant	a few kbps to 1 Mbps	yes, 100s of ms
Video conferencing	loss-tolerant	10 kbps to 5 Mbps	yes, 100s of ms
Stored audio/video	loss-tolerant	as above	yes, a few seconds
Interactive games	loss-tolerant	a few kbps to 10 kbps	yes, 100s of ms

Elastic means the application uses whatever throughput it finds: more is better, less still works.

TCP and UDP

The Internet makes exactly two transport protocols available, and you pick one.

- TCP: connection-oriented: a handshake precedes the data, so both ends are ready before anything flows.
- It provides reliable data transfer: the byte stream arrives complete, in order, with nothing missing and nothing duplicated.
- It also throttles the sender when the network is congested — for the good of the network, not of your application.
- UDP: connectionless: no handshake, no guarantee of arrival, no guarantee of order, no throttling.

UDP is not a broken TCP. It is what remains once you refuse to pay for the guarantees.

What the Internet does not guarantee

Of the four services, today's Internet transport protocols provide one.

- No throughput guarantee and no timing guarantee: neither TCP nor UDP offers either of them.
- Time-sensitive applications work anyway because they are designed to cope, not because the network helps them.
- No security either: what you push into the socket travels as you wrote it, password included.

Where TLS (Transport Layer Security) sits

TLS is not a third transport protocol. It is an enhancement of TCP, implemented in the application layer: the application encrypts before the socket and decrypts after it.

What an application-layer protocol defines

Application-layer protocol

It defines how the processes of an application, running on different end systems, pass messages to each other.

1. The types of message exchanged — requests and responses, typically.
2. The syntax of each type: which fields there are, and how they are delimited.
3. The semantics of the fields: what the information in them means.
4. The rules: when a process sends a message, and how it answers one.

Some are public and published as RFCs; others, like WhatsApp's, are proprietary and deliberately closed.

A protocol is not the application

The Web is an application. HTTP is one piece of it.

- The Web also needs a document format — HTML — plus browsers and servers.
- HTTP fixes only the format and the order of the messages between browser and Web server.
- Same for video on demand: servers, billing, the app on your phone, and DASH (Dynamic Adaptive Streaming over HTTP) for the messages.

Why insist on the distinction?

Because the protocol is the only part both ends must agree on. Everything else each side builds as it likes.

Application protocols, and the transport they use

Protocol	What it does	Port	Transport
DHCP	assigns IP addresses	67, 68	UDP
DNS	translates names into addresses	53	UDP, TCP
HTTP / HTTPS	transfers Web pages	80 / 443	TCP
SMTP / SMTPS	sends mail messages	25 / 465	TCP
SNMP	manages network devices	161	UDP
Telnet / SSH	command line on a remote host	23 / 22	TCP
FTP / FTPS	transfers files	21 / 990	TCP

Some of these carry credentials and personal data, so each has a secured variant beside it. On a public network the plain one is a mistake.

FTP

FTP, the File Transfer Protocol, is one of the oldest protocols on the Internet — the first version dates from 1971 — and it moves files between hosts.

- In its basic form everything travels in clear text: user name, password and file contents.
- So plain FTP belongs on a local or private network, and nowhere else.
- FTPS wraps it in TLS: credentials protected, contents encrypted.
- Like every application it is two things: the protocol, and the programs that implement it on each side.

FTP uses two connections

Unlike the protocols we have met so far, FTP does not carry its commands and its data on the same connection.

- The control connection, on port 21, opens first and stays open for the whole session: commands and replies only.
- Every transfer — a `get`, a `put`, even an `ls` — opens a second connection for the data alone, and closes it at the end.
- Control kept apart from data is called out-of-band signalling.

Who opens the data connection?

In active mode the server connects back to the client — and a client behind a NAT or a firewall cannot be reached from outside. Passive mode exists for exactly this reason: there the client opens both.

An FTP session

On the server a daemon waits in the background for connections; on the client, the `ftp` program.

```
$ sudo apt install vsftpd            server side, once
$ systemctl status vsftpd            is the daemon running?

$ ftp 143.225.161.30                 client side
Name: student
Password:
230 Login successful.
ftp> ls                              from here on, the ftp> prompt
ftp> get report.pdf                  server to client
226 Transfer complete.
ftp> bye                             bye or quit, not exit
```

Nearly every server you will write in this course has this shape: a daemon, waiting. FTP sends the password in clear: today files travel over SFTP, inside SSH, or over HTTPS.

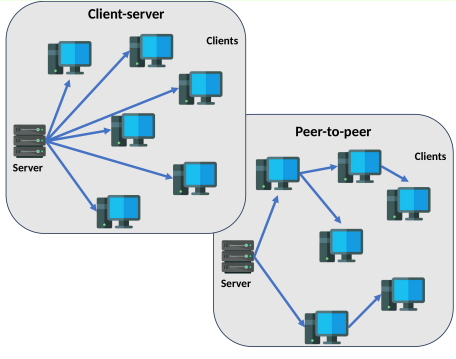
FTP commands

Cmd	Meaning	Cmd	Meaning
`help`	list the available commands	`pwd`	print the remote directory
`ls`	list the remote directory	`mkdir`	create a remote directory
`cd`	change the remote directory	`rmdir`	remove a remote directory
`lcd`	change the local directory	`delete`	delete a remote file
`get`	copy a file server to client	`put`	copy a file client to server

Two of them are the reason the protocol exists; the rest is navigation. Note `lcd`: the only one that acts on your own machine.

Distributing a file: two ways

One file, one server, and many hosts that want a copy of it.



On the left the server sends N copies. On the right it sends roughly one, and the peers pass the pieces on.

Why P2P scales

A file of F bits to N peers. The server uploads at u_s, peer i uploads at u_i, and d_{min} is the slowest download rate among the peers.

Minimum distribution time

$$D_{cs} = \max\left\{\frac{NF}{u_s},\ \frac{F}{d_{min}}\right\} \qquad D_{P2P} = \max\left\{\frac{F}{u_s},\ \frac{F}{d_{min}},\ \frac{NF}{u_s + \sum_{i=1}^{N} u_i}\right\}$$

- Client-server: the server alone must send NF bits, so the time grows linearly with N, without bound.
- P2P: each new peer adds its u_i to the denominator as well as an F to the numerator.

A thousand times the peers is a thousand times the wait — or almost no wait at all.

BitTorrent: torrents and chunks

Torrent

The collection of all the peers taking part in the distribution of one particular file.

- The file is cut into equal-size chunks, typically 256 kbytes, and peers trade chunks with one another.
- A peer joins with nothing, starts accumulating, and uploads while it is still downloading.
- Once it has the whole file it may leave, or stay and keep serving the others.
- It may also leave with only some of the chunks and rejoin later: the torrent tolerates that.

The tracker

Each torrent has one infrastructure node, the tracker — which is a server, in an architecture that was supposed not to need one.

- A new peer registers with it, and keeps telling it periodically that it is still there.
- In return it receives the IP addresses of a randomly chosen subset of the participating peers.
- It opens concurrent TCP connections to all of them: those that answer become its neighbours.
- Neighbours come and go, and peers outside the initial list may connect to it in turn.

Then it periodically asks each neighbour which chunks it has, and decides on that.

Rarest first, and tit-for-tat

Two decisions, taken over and over: which chunk to ask for, and whom to serve.

- Rarest first: among the chunks it lacks, a peer asks for the one fewest of its neighbours have — so rare chunks spread fast and the copies stay even.
- Trading: it serves the four neighbours currently sending it data fastest, recomputed every 10 seconds.
- And one at random: every 30 seconds it also serves one further neighbour, picked by chance.

Why the random one

Without it a peer holding nothing would never be served, and could never get started. It is also how peers with matching upload rates find each other.

Summary

- A network application is programs on end systems: nothing of it runs in the core.
- The developer picks an architecture — client-server, or P2P and its self-scalability.
- What communicates are processes, through sockets, and the client is whoever speaks first.
- Reaching one takes an address for the host and a port for the process: the network layer and the transport layer, one each.
- The transport offers reliability, and not throughput, timing or security: the application copes with the rest.
- An application-layer protocol fixes messages, syntax, meaning and rules — and is only one piece of an application.

Next we take one of these protocols apart, the one you use every day.